

SQL VIEW — Complete Revision Notes

Comprehensive Exam & Interview Guide | Based on Gate Smashers Core Concepts

🎯 INTERVIEW ANSWER — ONE-LINER DEFINITION

"A View is a virtual table created from a stored query that presents a controlled window of data from one or more base tables without storing the data physically."

1. CORE CONCEPTS & ARCHITECTURE

FUNDAMENTALS

WHAT IS A VIEW?

A View is a **virtual table** derived from one or more physical tables. When queried, it looks and behaves like an ordinary table, but it lacks physical storage for raw records.

Memory Trick:

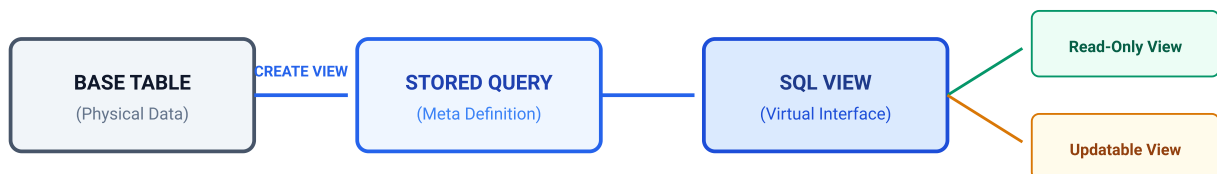
- **Base Table:** Stores actual raw data on disk.
- **View:** Stores the *query definition* in the data dictionary.

BASE TABLE RELATIONSHIP

The original physical table from which a view extracts data is called the **Base Table**.

Golden Rule: The View dynamic reflects base table data at the exact moment of execution. If underlying data changes, the view immediately reflects those updates.

2. VIRTUAL FLOW & DDL/DML MECHANICS



Basic View Syntax:

```
CREATE VIEW view_name AS SELECT column1, column2 FROM table_name WHERE condition;
```

Querying a View:

```
SELECT * FROM view_name; SELECT column1 FROM view_name WHERE condition;
```

⚠️ VIEW AND DDL/DML OPERATIONS

- **DDL (Data Definition Language):** Structural alterations made to the view definition do not alter the base table layout directly. Dropping a view removes only the virtual definition, not the base table or its data.
- **DML (Data Manipulation Language):** Permitted INSERT, UPDATE, and DELETE operations executed on an *Updatable View* directly modify rows in the underlying base table.

3. VIEW CAPABILITIES & MODIFICATIONS

CORE MECHANICS

FILTERING ROWS & COLUMNS

- **Reduce Columns:** Expose only required fields (e.g., exposing 2 columns out of a 10-column table).
- **Filter Rows:** Use a WHERE clause to restrict accessible records (e.g., filtering students by city).

```
CREATE VIEW Delhi_Students AS SELECT ID, Name, Course FROM Student WHERE Address = 'Delhi';
```

MULTI-TABLE VIEWS (JOINS)

Views can aggregate fields across multiple tables using SQL JOINS or subqueries seamlessly.

```
CREATE VIEW Student_Course_VW AS SELECT S.ID, S.Name, C.CourseName FROM Student S JOIN Enrollment E ON S.ID = E.StudentID JOIN Course C ON E.CourseID = C.ID;
```

4. BASE TABLE DATA REFLECTION EXAMPLE

Demonstration of how Base Table mutations reflect live in the corresponding View:

1. Base Table: Student

ID	Name	Address
1	Steven	Delhi
2	John	Mumbai

2. Created View: V1

ID	Name
1	Steven
2	John

BASE TABLE MUTATION TEST

If we execute: `UPDATE Student SET Name = 'Steven Smith' WHERE ID = 1;`

Querying `SELECT * FROM V1;` will output **Steven Smith** for ID 1. Deleting or inserting a row in Student similarly adds or removes rows visible in V1.

5. NORMAL VIEW VS. MATERIALIZED VIEW

HIGH FREQUENCY INTERVIEW QUESTION

Feature / Property	Normal View (Virtual)	Materialized View (Snapshot)
Physical Storage	None (Stores query definition only)	Yes (Stores query result data physically)
Performance	Executes query every time view is accessed	Fast read access (reads local disk snapshot)
Data Freshness	Always real-time / up-to-date	Stale until explicitly refreshed
Use Case	Security, query abstraction, dynamic data	Data warehousing, remote servers, heavy analytics



MATERIALIZED VIEW CONCEPT

Useful when pulling data over a network or remote server. Instead of fetching live data across slow network

6. FOUR MAJOR ADVANTAGES OF VIEWS

CORE THEORY

1. RESTRICT DATA ACCESS (SECURITY) 🔒

Views shield sensitive base-table columns (e.g., Salary, Password, Mobile) from unauthorized users, exposing only required information.

2. SIMPLIFY COMPLEX QUERIES 🧩

Encapsulates heavy JOINS, multi-stage aggregations, and complex WHERE logic into a simple virtual table for repetitive use.

3. DATA INDEPENDENCE 🛡️

Provides a structural abstraction layer. Schema modifications in base tables can be smoothed over by altering the view definition.

4. USER-SPECIFIC CUSTOMIZATION 👥

Allows different security roles (Student, Faculty, Admin) to see tailored perspectives of the same underlying database tables.

7. REAL-LIFE UNIVERSITY DATABASE CASE STUDY

Scenario: A university database maintains a unified Faculty base table with sensitive fields:

Faculty (ID, Name, Qualification, Address, Phone, Salary, BankAccount)

Requirement: Students need to check faculty qualifications for course selection without exposing salary or private contact info.

Solution: Construct a public-facing view:

```
CREATE VIEW Public_Faculty_Info AS SELECT Name, Qualification FROM Faculty;
```

8. IMPORTANT EXAM TRAPS & CONCEPTUAL CHECKLIST

⚠️ COMMON EXAM TRAPS

- Q: Is a view physically created on disk?**
A: No, it is a virtual construct.
- Q: Does dropping a view delete base table data?**
A: No, only the query metadata is destroyed.
- Q: Can all views perform INSERT/DELETE?**
A: No. Views with aggregations, DISTINCT, or GROUP BY are usually Read-Only.

⚡ 30-SECOND FINAL CHECKLIST

Normal View	Virtual / Query stored
Materialized View	Physical / Data stored
Updatable View	DML updates Base Table
Base Table	Primary physical source

🎯 FINAL SUMMARY FOR ORAL VIVAS & INTERVIEWS

"Views enforce row/column-level security, simplify multi-table query logic, provide logical data independence, and allow multiple user-tailored representations over a single physical database schema."